

All COs Assessment

Team No:11

Team members: R YASWANTH REDDY -192424337

NARESH M – 192424381

Course code: DSA0503

Subject : Query Processing for Data Science

Title: Student Academic Performance Data Wrangling and Visualization System

Problem Statement

A university maintains student academic information from multiple sources such as admission, course registration, attendance, examination, and placement systems. Since these datasets are collected independently and stored in CSV, JSON, and XML formats, they may contain missing values, duplicates, inconsistent names, invalid values, and outliers.

The proposed system integrates these datasets using Python, identifies and resolves data-quality problems, performs data wrangling and outlier treatment, and generates meaningful static and interactive visualizations to support academic decision-making.

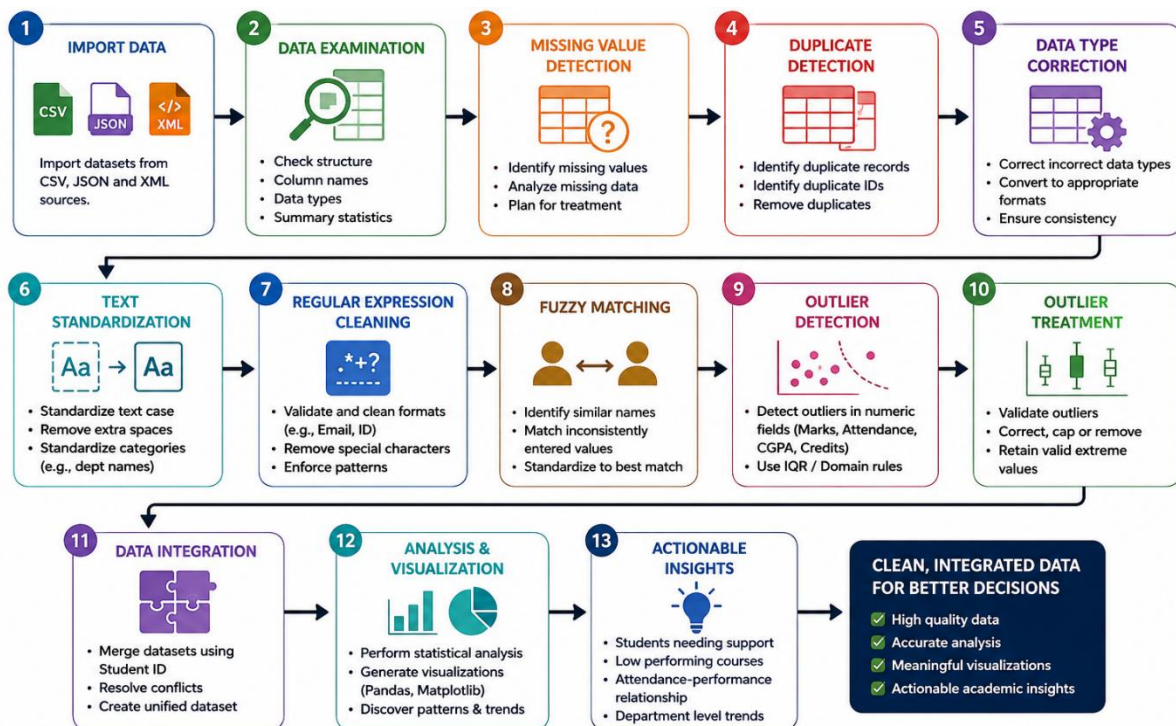
Objectives

- Import CSV, JSON, and XML datasets.
- Examine dataset structure and data types.
- Identify missing and duplicate records.
- Clean inconsistent textual data.
- Use regular expressions for formatting validation and cleaning.
- Apply fuzzy matching to identify similar student names.
- Detect and treat numerical outliers.
- Integrate data from multiple sources.
- Generate visualizations using Pandas and Matplotlib.
- Provide interactive academic visualizations.
- Generate actionable academic insights.

Input Data

Source	Format	Important Attributes
Admission	CSV	Student ID, Name, Department, Email
Registration	JSON	Student ID, Course, Credits, Semester
Attendance	XML	Student ID, Course, Attendance
Examination	CSV	Student ID, Course, Marks, Grade
Placement	JSON	Student ID, Company, Placement Status

Data Wrangling process:



Algorithm:

START

1. Load student datasets from CSV, JSON and XML files.
2. Examine each dataset:
 - a. Identify rows and columns.
 - b. Check data types.
 - c. Check missing values.
 - d. Check duplicate records.
3. Clean the datasets:

- a. Handle missing values.
 - b. Remove duplicate records.
 - c. Correct data types.
 - d. Standardize names and categories.
4. Apply regular expressions:
 - a. Validate Student IDs.
 - b. Validate email formats.
 - c. Remove unwanted characters and spaces.
 5. Apply fuzzy matching:
 - a. Compare similar student names.
 - b. Calculate similarity scores.
 - c. Identify possible duplicate/inconsistent names.
 6. Detect numerical outliers using the IQR method.
 7. Validate values using domain rules:

Marks → 0–100

Attendance → 0–100

CGPA → 0–10
 8. Treat invalid and potential outlier values appropriately.
 9. Merge the cleaned datasets using Student_ID.
 10. Perform academic analysis.
 11. Generate visualizations using Pandas and Matplotlib.
 12. Generate interactive visualizations using the dashboard.
 13. Identify actionable academic insights
 14. Generate and export cleaned datasets and reports.
- END

Data Quality Issues

The system should identify:

- Missing values

- Duplicate records
- Inconsistent student names
- Inconsistent department names
- Invalid email formats
- Invalid student IDs
- Invalid marks
- Invalid attendance percentages
- Invalid CGPA values
- Numerical outliers
- Different data formats across sources

Data Cleaning Techniques

Issue	Technique
Missing values	Mean/median/mode or suitable replacement
Duplicates	drop_duplicates()
Names	Case and whitespace standardization
Email	Regular expression
Student ID	Regular expression
Department	Mapping/standardization
Similar names	Fuzzy matching
Invalid marks	Domain validation
Outliers	IQR method

Fuzzy Matching Example

Original Data:

- 1.Arun Kumar
- 2.ARUN KUMAR
- 3.ArunKumar

4.Arun Kumr

Output :

Arun Kumar

fuzzy matching should identify and referring to the same student and standardizes it.

Outlier Detection

For numerical attributes such as **marks, attendance, credits, and CGPA**, the system uses the IQR method

$$\text{IQR} = Q3 - Q1$$

$$\text{Lower Limit} = Q1 - 1.5 \times \text{IQR}$$

$$\text{Upper Limit} = Q3 + 1.5 \times \text{IQR}$$

Rules:

Marks → 0–100

Attendance → 0–100%

CGPA → 0–10

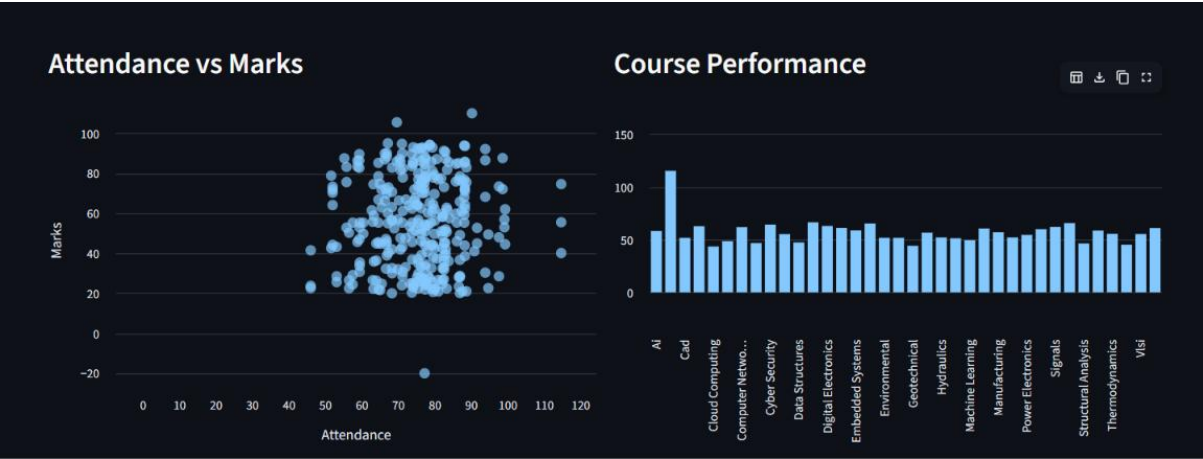
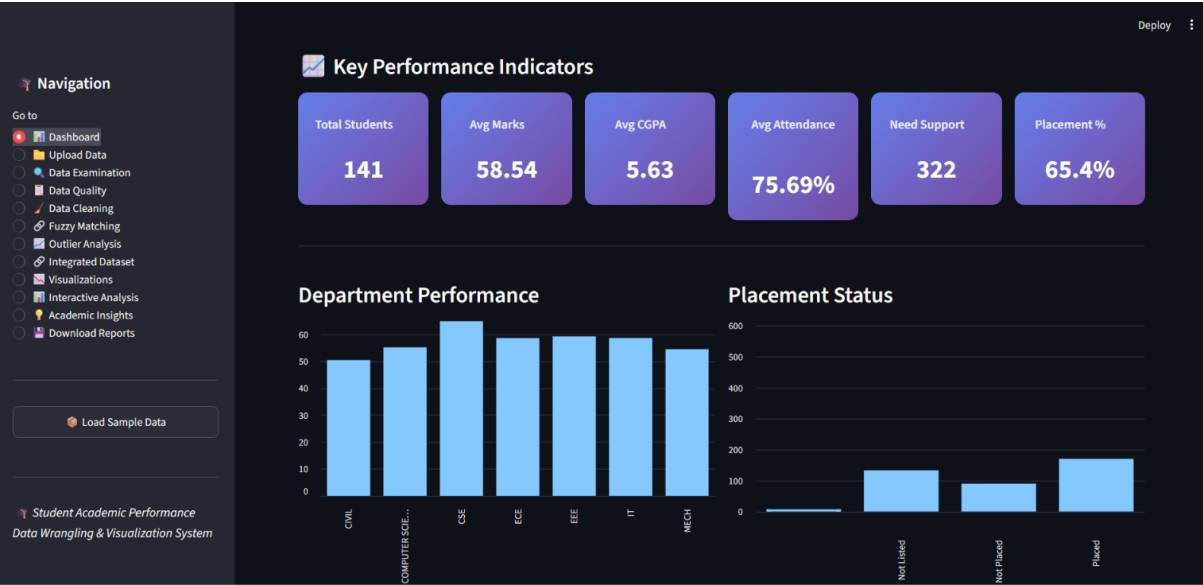
Credits → Positive values

Visualization

The system should generate:

1. **Marks Distribution** – Histogram
2. **CGPA Distribution** – Histogram
3. **Department-wise Performance** – Bar chart
4. **Course-wise Average Marks** – Bar chart
5. **Attendance vs Marks** – Scatter plot
6. **Semester Performance Trend** – Line chart
7. **Placement vs CGPA** – Comparison chart

Visualization:



Students Requiring Academic Support

	Student_ID	Name	Department	CGPA	Avg_Marks	Attendance	Reason
0	S047	Riya Malhotra	CIVIL	2.67	26.73	77.31	Low CGPA and Low Marks
1	S029	Aakash Chauhan	CSE	2.85	28.5	71.25	Low CGPA and Low Marks and Low Attendance
2	S074	Rahul Reddy	EEE	2.93	29.27	46.08	Low CGPA and Low Marks and Low Attendance
3	S011	Arun Kumar	CSE	3.24	32.45	78	Low CGPA and Low Marks
4	S088	Ashish Gupta	IT	3.24	32.43	53.2	Low CGPA and Low Marks and Low Attendance
5	S099	Prakash Mishra	ECE	3.24	32.4	75.62	Low CGPA and Low Marks
6	S149	Suresh Bhatt	CIVIL	3.3	33.02	None	Low CGPA and Low Marks
7	S132	Sneha Chauhan	CSE	3.31	33.1	56.6	Low CGPA and Low Marks and Low Attendance
8	S130	Megha Das	IT	3.32	33.2	76.34	Low CGPA and Low Marks
9	S049	Karthik Das	CSE	3.32	33.25	None	Low CGPA and Low Marks

Showing top 15 of 107 students requiring support.

Navigation

Go to

- Dashboard
- Upload Data
- Data Examination
- Data Quality
- Data Cleaning
- Fuzzy Matching
- Outlier Analysis
- Integrated Dataset
- Visualizations
- Interactive Analysis
- Academic Insights
- Download Reports

Load Sample Data

Student Academic Performance
Data Wrangling & Visualization System

Data Examination

Select Dataset

- admission

Total Rows

155

Total Columns

6

Duplicate Rows

1

Column Details

	Column	Data Type	Missing Count	Missing %	Unique Values
0	Student_ID	str	0	0.0%	150
1	Name	str	11	7.1%	137
2	Gender	str	2	1.3%	2
3	Department	str	3	1.9%	21
4	Email	str	5	3.2%	145
5	Admission_Year	int64	0	0.0%	5

Statistical Summary

	Student_ID	Name	Gender	Department	Email	Admission_Year
--	------------	------	--------	------------	-------	----------------

Deploy

Navigation

Go to

- Dashboard
- Upload Data
- Data Examination
- Data Quality
- Data Cleaning
- Fuzzy Matching
- Outlier Analysis
- Integrated Dataset
- Visualizations
- Interactive Analysis
- Academic Insights
- Download Reports

Load Sample Data

Student Academic Performance
Data Wrangling & Visualization System

	Student_ID	Name	Gender	Department	Email	Admission_Year
count	155	144	153	152	150	155
unique	150	137	2	21	145	None
top	S006	Ananya Gupta	Female	IT	lakshmi.nair@yahoo.cc	None
freq	2	2	80	28	2	None
mean	None	None	None	None	None	2019.98
std	None	None	None	None	None	1.38
min	None	None	None	None	None	2018
25%	None	None	None	None	None	2019
50%	None	None	None	None	None	2020
75%	None	None	None	None	None	2021

Data Types & Non-Null Info

	Column	Non-Null Count	Dtype	Null Count	Unique
0	Student_ID	155	str	0	150
1	Name	144	str	11	137
2	Gender	153	str	2	2
3	Department	152	str	3	21
4	Email	150	str	5	145

Deploy

Navigation

Go to

- Dashboard
- Upload Data
- Data Examination
- Data Quality
- Data Cleaning
- Fuzzy Matching
- Outlier Analysis
- Integrated Dataset
- Visualizations
- Interactive Analysis
- Academic Insights
- Download Reports

Load Sample Data

Student Academic Performance
Data Wrangling & Visualization System

Data Preview (First 20 Rows)

	Student_ID	Name	Gender	Department	Email	Admission_Year
0	S001	Meera Kumar	Male	IT	meera.kumar@college.edu	2019
1	S002	Sneha Reddy	Female	ECE	sneha.reddy@gmail.com	2018
2	S003	Ananya Gupta	Male	CSE	ananya.gupta@yahoo.com	2022
3	S004	Komal Bhatt	Male	None	komal.bhatt@hotmail.com	2021
4	S005	Lakshmi Rao	Male	CIVIL	lakshmi.rao@gmail.com	2021
5	S006	None	Female	EEE	lakshmi.nair@yahoo.com	2018
6	S007	Nitin Reddy	Female	CSE	nitin.reddy@college.edu	2022
7	S008	Rahul Tiwari	None	EEE	rahul.tiwari@hotmail.com	2021
8	S009	Megha Verma	Male	CSE	None	2018
9	S010	Sanjay Joshi	Male	CSE	sanjay.joshi@college.edu	2019

5 | Admission_Year | 155 | int64 | 0 | 5

Deploy

Navigation

Go to

- Dashboard
- Upload Data
- Data Examination
- Data Quality**
- Data Cleaning
- Fuzzy Matching
- Outlier Analysis
- Integrated Dataset
- Visualizations
- Interactive Analysis
- Academic Insights
- Download Reports

Student Academic Performance

Data Wrangling & Visualization System

Deploy

Data Quality Report

	Dataset	Rows	Columns	Missing Values	Duplicate Rows	Duplicate IDs	Status
0	admission	155	6	21	1	5	Loaded
1	registration	302	4	0	3	180	Loaded
2	attendance	251	3	6	1	131	Loaded
3	examination	402	5	11	2	261	Loaded
4	placement	101	4	0	1	1	Loaded

Detailed View

admission

Navigation

Go to

- Dashboard
- Upload Data
- Data Examination
- Data Quality**
- Data Cleaning
- Fuzzy Matching
- Outlier Analysis
- Integrated Dataset
- Visualizations
- Interactive Analysis
- Academic Insights
- Download Reports

Student Academic Performance

Data Wrangling & Visualization System

admission

Missing Values in admission

	Column	Missing Count	Missing %
0	Student_ID	0	0.0%
1	Name	11	7.1%
2	Gender	2	1.3%
3	Department	3	1.9%
4	Email	5	3.2%
5	Admission_Year	0	0.0%

Duplicate Analysis in admission

Exact Duplicate Rows

1

Duplicate Student IDs

5

Navigation

Go to

- Dashboard
- Upload Data
- Data Examination
- Data Quality**
- Data Cleaning
- Fuzzy Matching
- Outlier Analysis
- Integrated Dataset
- Visualizations
- Interactive Analysis
- Academic Insights
- Download Reports

Student Academic Performance

Data Wrangling & Visualization System

Deploy

View Cleaned Dataset

admission

	Student_ID	Name	Gender	Department	Email	Admission_Year
0	S001	Meera Kumar	Male	IT	meera.kumar@college.edu	2019
1	S002	Sneha Reddy	Female	ECE	sneha.reddy@gmail.com	2018
2	S003	Ananya Gupta	Male	CSE	ananya.gupta@yahoo.com	2022
3	S004	Komal Bhatt	Male	CSE	komal.bhatt@hotmail.com	2021
4	S005	Lakshmi Rao	Male	CIVIL	lakshmi.rao@gmail.com	2021
5	S006	Ananya Gupta	Female	EEE	lakshmi.nair@yahoo.com	2018
6	S007	Nitin Reddy	Female	CSE	nitin.reddy@college.edu	2022
7	S008	Rahul Tiwari	Female	EEE	rahul.tiwari@hotmail.com	2021
8	S009	Megha Verma	Male	CSE	lakshmi.nair@yahoo.com	2018
9	S010	Sanjay Joshi	Male	CSE	sanjay.joshi@college.edu	2019

Navigation

Go to

Dashboard

Upload Data

Data Examination

Data Quality

Data Cleaning

Fuzzy Matching

Outlier Analysis

Integrated Dataset

Visualizations

Interactive Analysis

Academic Insights

Download Reports

Load Sample Data

Student Academic Performance

Data Wrangling & Visualization System

Deploy

Similarity Threshold

85

Select Dataset for Fuzzy Matching

admission

Found 3 Potential Name Variants

	Original Name	Possible Match	Similarity
0	Karthik Singh	Karthik Sinha	92.3
1	Mohit Sinha	Mohit Singh	90.9
2	Kavitamalhotra	Kavita Malhotra	89.7

Apply Name Standardization (Select Canonical Names)

Navigation

Go to

Dashboard

Upload Data

Data Examination

Data Quality

Data Cleaning

Fuzzy Matching

Outlier Analysis

Integrated Dataset

Visualizations

Interactive Analysis

Academic Insights

Download Reports

Load Sample Data

Student Academic Performance

Data Wrangling & Visualization System

Outlier Analysis

Detect statistical outliers (IQR method) and domain rule violations.

IQR Analysis Summary

	Dataset	Column	Q1	Q3	IQR	Lower Bound	Upper Bound
0	registration	Credits	3	4	1	1.5	5.5
1	attendance	Attendance	65	86.7	21.7	32.45	119.25
2	examination	Marks	36.62	75.9	39.28	-22.29	134.81
3	placement	Package_LPA	0	29.38	29.38	-44.06	73.44

Run Full Outlier Detection

Outlier Report (14 entries)

	Student_ID	Attribute	Value	Outlier_Status	Treatment	Dataset
0	S128	Credits	-3	Invalid (Domain violation)	Convert to NaN	registration
1	S011	Credits	0	Invalid (Domain violation)	Convert to NaN	registration
2	S074	Attendance	-10	Invalid (Domain violation)	Convert to NaN	attendance
3	S012	Attendance	200	Invalid (Domain violation)	Convert to NaN	attendance

Navigation

Go to

Dashboard

Upload Data

Data Examination

Data Quality

Data Cleaning

Fuzzy Matching

Outlier Analysis

Integrated Dataset

Visualizations

Interactive Analysis

Academic Insights

Download Reports

Load Sample Data

Student Academic Performance

Data Wrangling & Visualization System

Deploy

0	S094	Marks	999	Invalid (Domain violation)	Convert to NaN	examination
7	S047	Marks	-20	Invalid (Domain violation)	Convert to NaN	examination
8	S085	Marks	110	Invalid (Domain violation)	Convert to NaN	examination
9	S031	Marks	105.5	Invalid (Domain violation)	Convert to NaN	examination
10	S066	Semester	0	Invalid (Domain violation)	Convert to NaN	examination
11	S009	Semester	-1	Invalid (Domain violation)	Convert to NaN	examination
12	S078	Package_LPA	1000	Invalid (Domain violation)	Convert to NaN	placement
13	S016	Package_LPA	-5	Invalid (Domain violation)	Convert to NaN	placement

Domain Validation Rules

	Attribute	Min Valid	Max Valid
0	Marks		100
1	Attendance		100
2	CGPA		10
3	Credits		10
4	Semester		12
5	Package_LPA		100

Navigation

Go to

Dashboard

Upload Data

Data Examination

Data Quality

Data Cleaning

Fuzzy Matching

Outlier Analysis

Integrated Dataset

Visualizations

Interactive Analysis

Academic Insights

Download Reports

Load Sample Data

Student Academic Performance

Data Wrangling & Visualization System

Deploy

Data Integration

Merge all cleaned datasets into one unified student academic dataset using Student_ID.

Datasets Available for Integration

admission: 150 rows × 6 columns

registration: 299 rows × 4 columns

attendance: 250 rows × 3 columns

examination: 400 rows × 5 columns

placement: 100 rows × 4 columns

Integrate All Datasets

Current Integrated Dataset

	Student_ID	Name	Department	Email	Gender	Admission_Year	Course	Credits	Semester	Attendance	Mark
0	S113	Karthik Sinha	EEE	karthik.sinha@gmail.com	Male	2020	Machines	None	3	71.35	77.1
1	S063	Megha Nair	CIVIL	megha.nair@hotmail.com	Male	2018	Structural Analysis	None	5	68.4	56.2
2	S036	Raviyer	ECE	gaurav.nair@hotmail.com	Male	2021	Signals	None	3	78.2	28.1
3	S061	Pooja Choudhary	ECE	sneha.pandey@hotmail.com	Male	2018	Digital Electronics	4	4	58.9	45.1

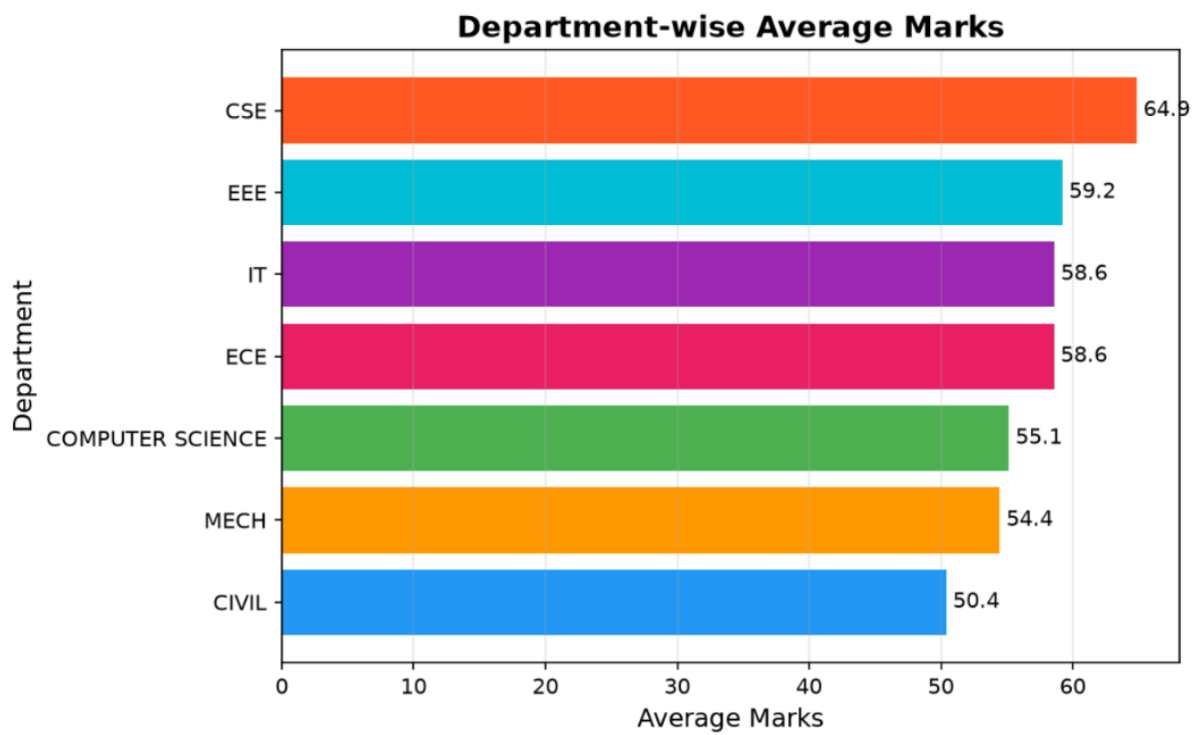
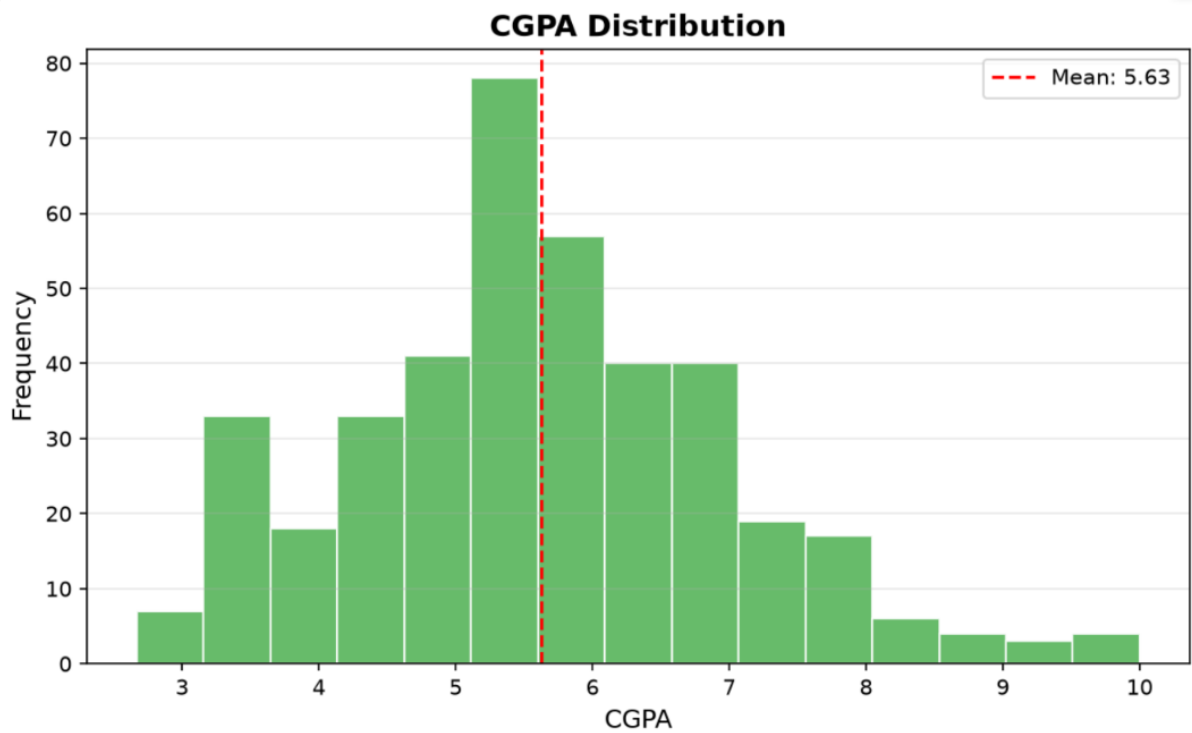
Marks Distribution

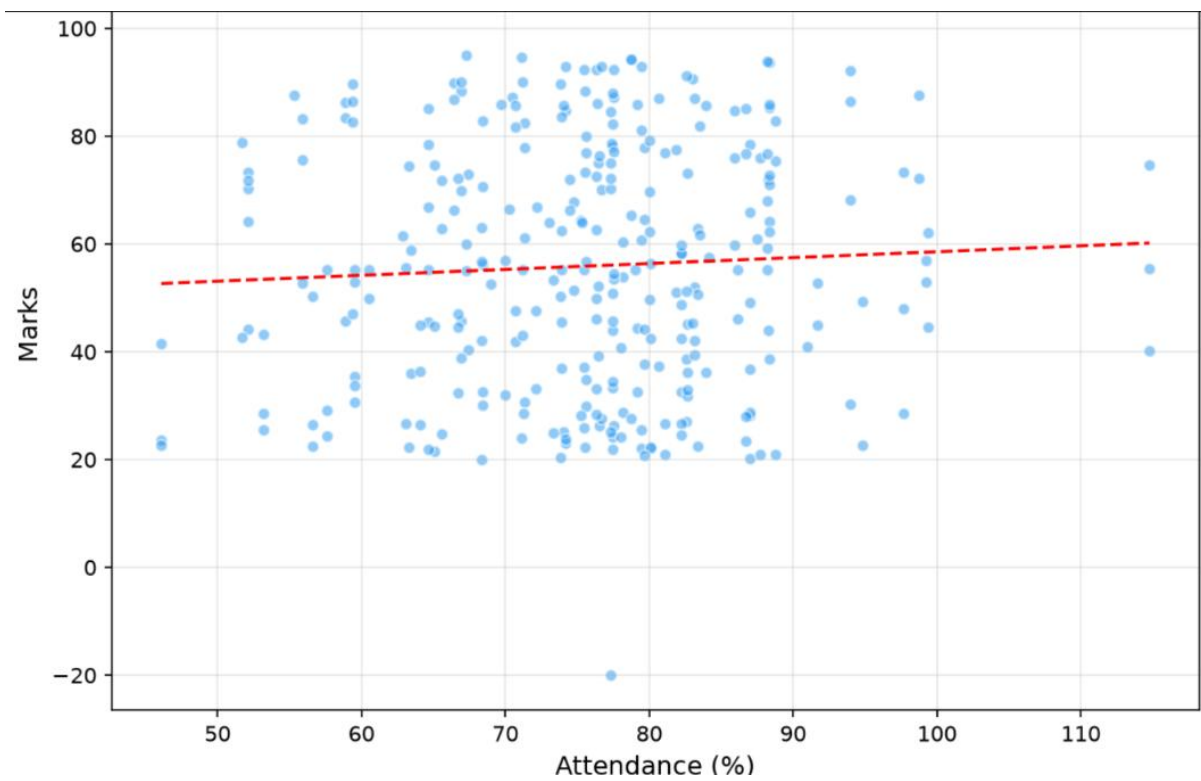
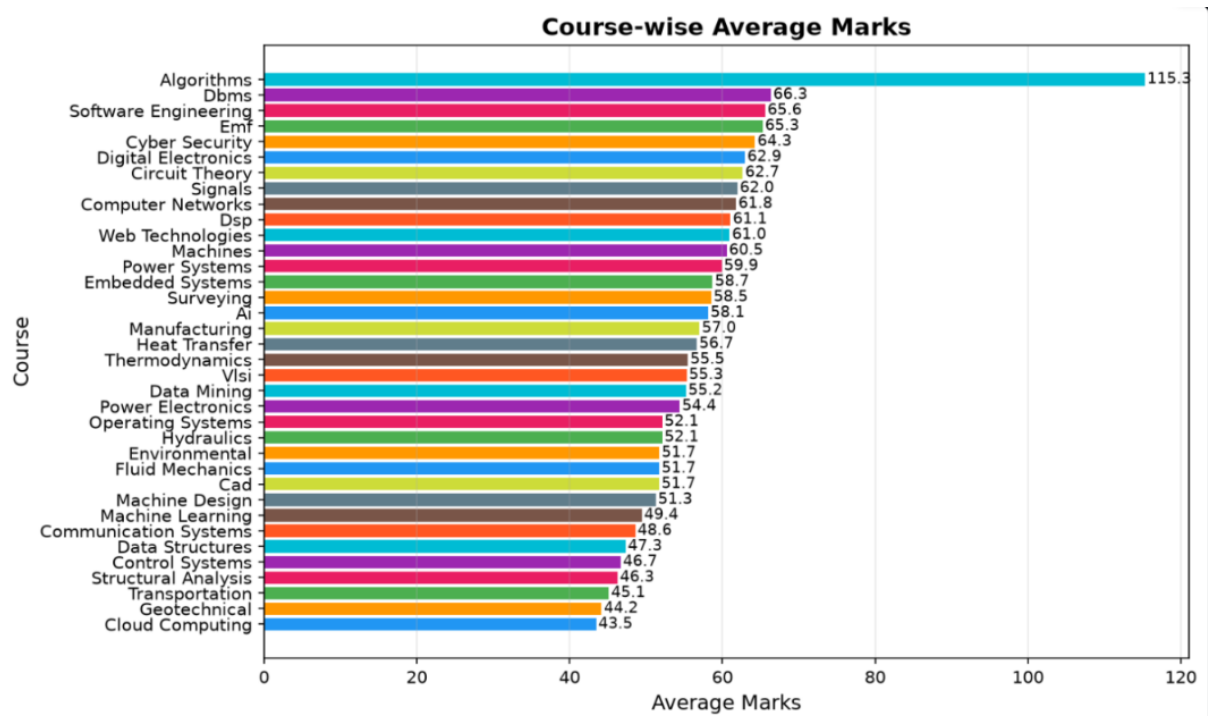
Frequency

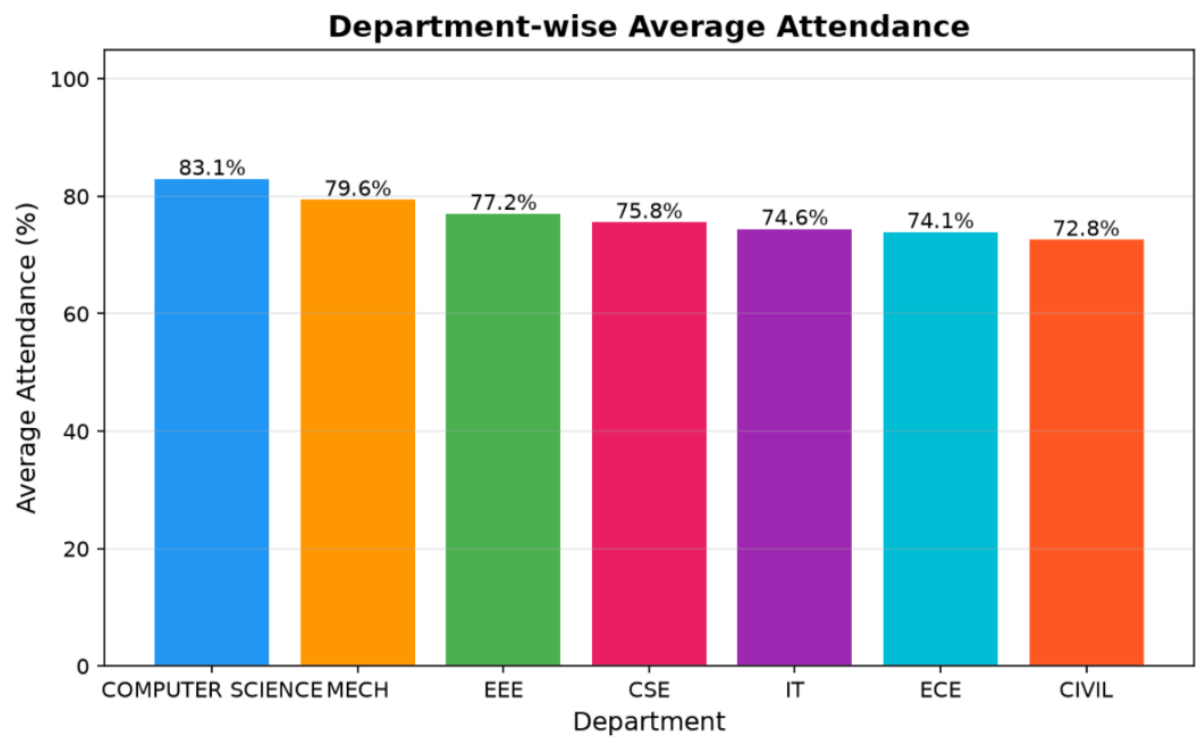
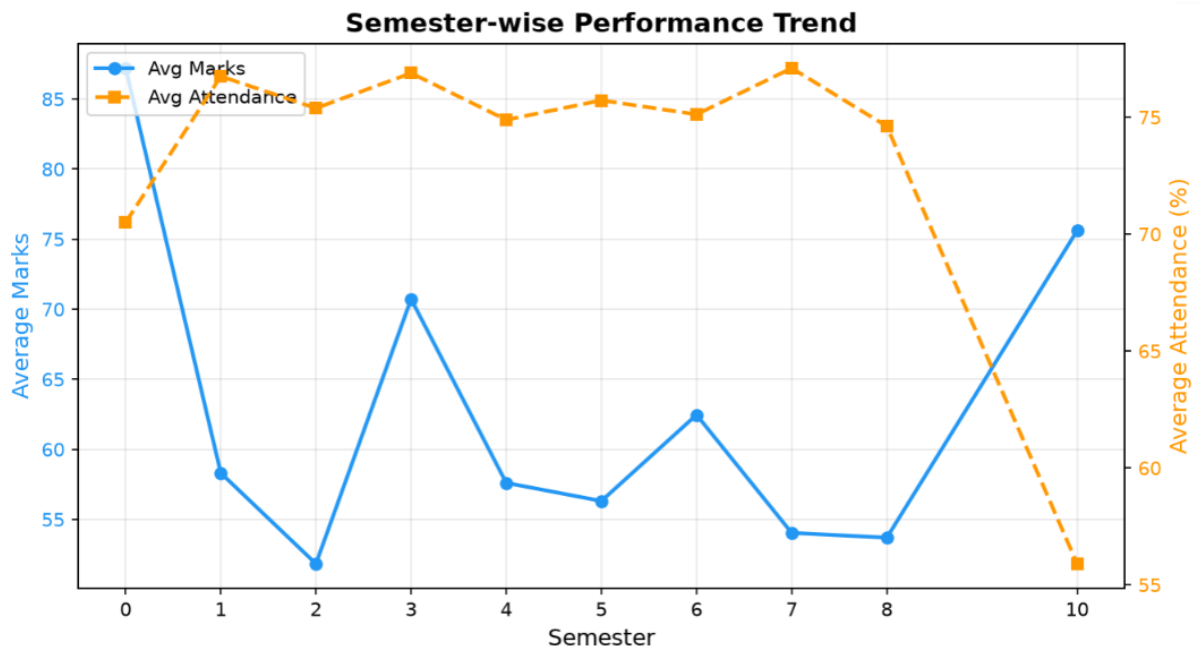
Marks

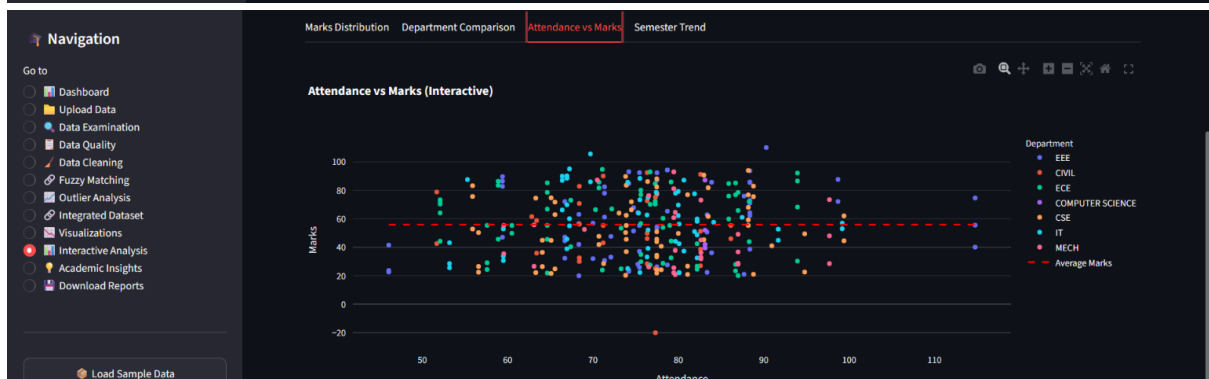
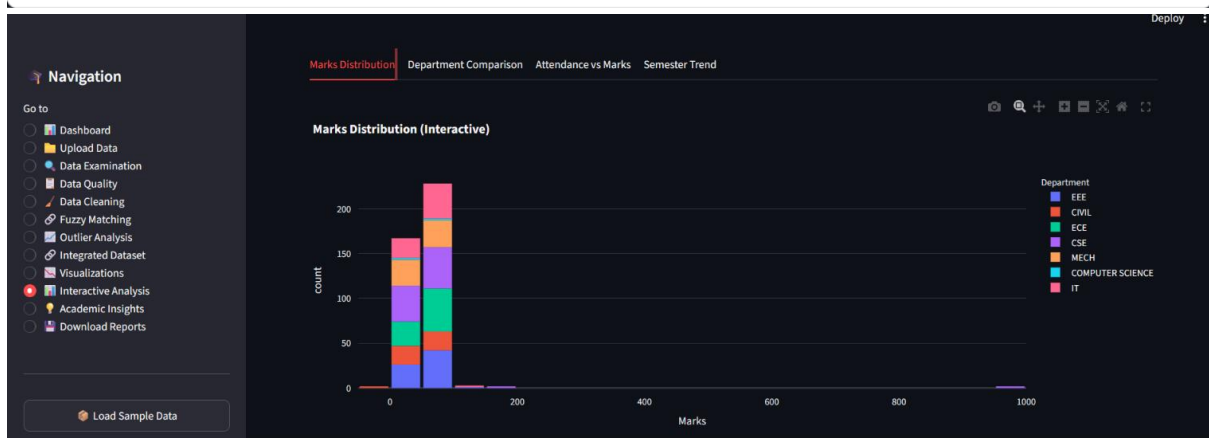
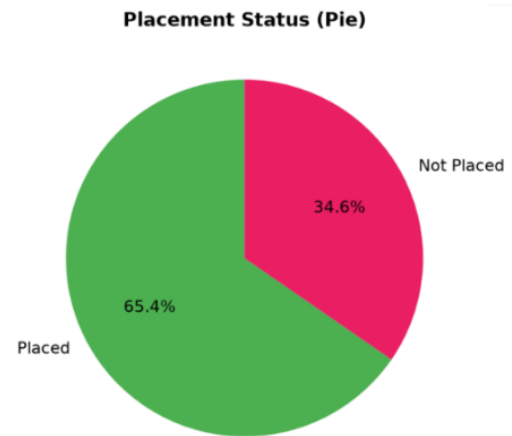
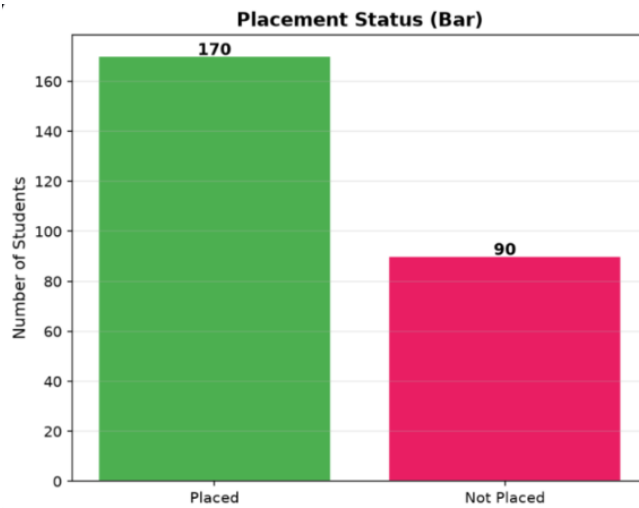
--- Mean: 58.5

Marks Range	Frequency
0 - 50	250
50 - 100	75
100 - 150	70









Recommendations:

- **Cloud Computing** (Avg: 43.46): Additional tutorial sessions; Attendance monitoring; Strict attendance policy enforcement
- **Geotechnical** (Avg: 44.17): Additional tutorial sessions; Attendance monitoring
- **Transportation** (Avg: 45.13): Additional tutorial sessions; Attendance monitoring; Strict attendance policy enforcement
- **Structural Analysis** (Avg: 46.28): Additional tutorial sessions; Attendance monitoring; Strict attendance policy enforcement
- **Control Systems** (Avg: 46.68): Additional tutorial sessions; Attendance monitoring
- **Data Structures** (Avg: 47.32): Additional tutorial sessions; Attendance monitoring
- **Communication Systems** (Avg: 48.59): Additional tutorial sessions; Attendance monitoring
- **Machine Learning** (Avg: 49.44): Additional tutorial sessions; Attendance monitoring; Strict attendance policy enforcement
- **Machine Design** (Avg: 51.28): Supplementary practice materials; Peer tutoring sessions
- **Cad** (Avg: 51.71): Supplementary practice materials; Peer tutoring sessions
- **Fluid Mechanics** (Avg: 51.73): Supplementary practice materials; Peer tutoring sessions; Strict attendance policy enforcement
- **Environmental** (Avg: 51.74): Supplementary practice materials; Peer tutoring sessions
- **Hydraulics** (Avg: 52.11): Supplementary practice materials; Peer tutoring sessions; Strict attendance policy enforcement

Department Performance Comparison

	Department	Student_Count	Avg_Marks	Avg_Attendance	Avg_CGPA	Placement_%
0	CIVIL	43	50.4	72.78	5.04	90.3
1	COMPUTER SCIENCE	4	55.15	83.12	5.52	0
2	CSE	88	64.85	75.75	5.46	65.3
3	ECE	75	58.57	74.09	5.86	65.7
4	EEE	69	59.21	77.22	5.91	58.9
5	IT	62	58.59	74.64	5.86	77.5
6	MECH	59	54.45	79.59	5.44	51.1

Placement Analysis

Placement Rate

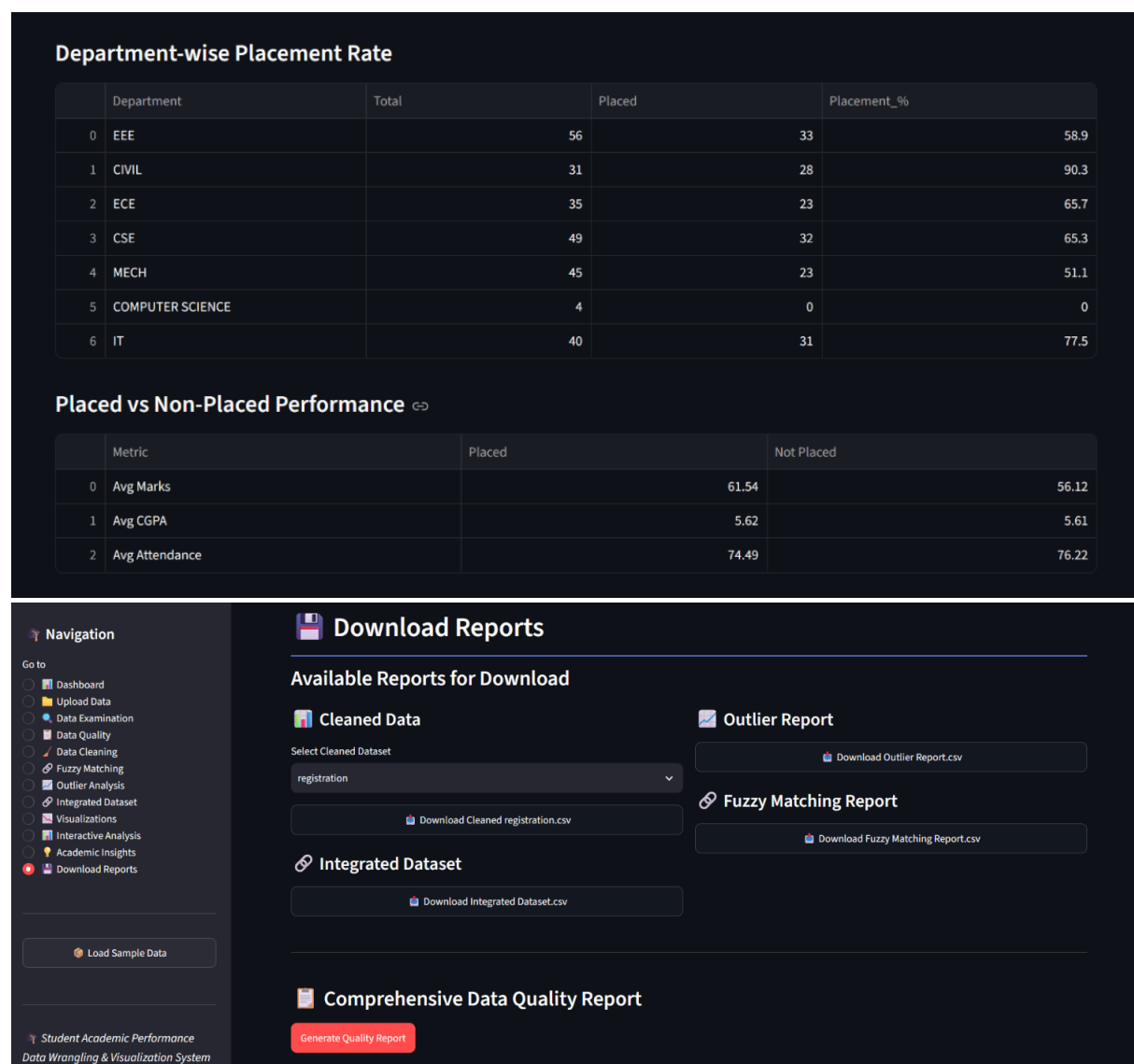
65.4%

Placed

170

Not Placed

90



```

import io

import tempfile

sys.path.insert(0, os.path.dirname(__file__))

from src.data_loader import detect_and_load, get_file_info

from src.data_profiler import profile_dataset, create_quality_summary_table, get_statistical_summary,
get_basic_info

from src.data_cleaner import full_clean_pipeline, standardize_name, standardize_department

from src.regex_cleaner import apply_regex_validation

from src.fuzzy_matcher import apply_fuzzy_matching, find_fuzzy_matches, create_standardization_map

from src.outlier_detector import detect_and_report_outliers, treat_outliers, detect_iqr_outliers,
DOMAIN_RULES

from src.data_integrator import integrate_datasets

from src.analyzer import (
    compute_basic_stats, department_analysis, course_analysis,
    semester_analysis, placement_analysis, attendance_marks_correlation,
    low_performing_courses, compute_kpis
)

from src.visualizer import generate_all_charts

from src.insights import find_students_needing_support, find_low_performing_courses,
attendance_performance_analysis

st.set_page_config(
    page_title="Student Academic Performance Analytics",
    page_icon="🎓",
    layout="wide",
    initial_sidebar_state="expanded",
)

st.markdown("""
<style>

.main-header { text-align: center; padding: 1rem 0; }
.main-header h1 { color: #1a237e; font-size: 2.2rem; }
.kpi-card {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white; padding: 1.2rem; border-radius: 12px;
    text-align: center; box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}
.kpi-card h3 { margin: 0; font-size: 1rem; opacity: 0.9; }
.kpi-card p { margin: 0.3rem 0 0 0; font-size: 2rem; font-weight: bold; }

```

```

        .section-header { color: #1a237e; border-bottom: 2px solid #667eea; padding-bottom: 0.3rem; }
</style>
"""', unsafe_allow_html=True)

def load_sample_data():
    """Load sample datasets from the data directory."""
    data_dir = os.path.join(os.path.dirname(__file__), "data")
    datasets = {}
    messages = {}
    for fname, dtype in [
        ("admission.csv", "csv"),
        ("registration.json", "json"),
        ("attendance.xml", "xml"),
        ("examination.csv", "csv"),
        ("placement.json", "json"),
    ]:
        fpath = os.path.join(data_dir, fname)
        if os.path.exists(fpath):
            df, ftype, msg = detect_and_load(fpath)
            key = fname.split(".")[0]
            datasets[key] = df
            messages[key] = msg
    return datasets, messages

def init_session_state():
    """Initialize Streamlit session state variables."""
    defaults = {
        "datasets": {},
        "cleaned_datasets": {},
        "integrated_df": None,
        "analysis_done": False,
        "fuzzy_report": pd.DataFrame(),
        "outlier_report": pd.DataFrame(),
        "validation_report": pd.DataFrame(),
        "cleaning_treatment": pd.DataFrame(),
        "charts": {},
    }
    for key, val in defaults.items():

```

```

        if key not in st.session_state:
            st.session_state[key] = val
    init_session_state()
    st.sidebar.markdown("## 🎓 Navigation")
    page = st.sidebar.radio(
        "Go to",
        [
            "📊 Dashboard",
            "📁 Upload Data",
            "🔍 Data Examination",
            "📄 Data Quality",
            "🧹 Data Cleaning",
            "🔗 Fuzzy Matching",
            "📈 Outlier Analysis",
            "🔗 Integrated Dataset",
            "📉 Visualizations",
            "📊 Interactive Analysis",
            "💡 Academic Insights",
            "📄 Download Reports",
        ],
    )
    st.sidebar.markdown("---")
    if st.sidebar.button("📦 Load Sample Data", use_container_width=True):
        with st.spinner("Loading sample datasets..."):
            datasets, messages = load_sample_data()
            st.session_state.datasets = datasets
            st.session_state.messages = messages
            st.sidebar.success("Sample data loaded!")
            for name, msg in messages.items():
                st.sidebar.caption(f"*** {name} ***: {msg}")
    if page == "📊 Dashboard":
        st.markdown('<div class="main-header"><h1>🎓 Student Academic Performance Analytics</h1></div>',
                    unsafe_allow_html=True)
        if not st.session_state.datasets:

```

```
st.info("👉 Click **Load Sample Data** in the sidebar or upload files from the **Upload Data** page to get started.")
```

```
st.stop()
```

```
available = {k: v for k, v in st.session_state.datasets.items() if v is not None and not v.empty}
```

```
if not available:
```

```
    st.warning("No valid datasets loaded.")
```

```
    st.stop()
```

```
if st.session_state.integrated_df is not None and not st.session_state.integrated_df.empty:
```

```
    df = st.session_state.integrated_df
```

```
else:
```

```
    # Quick merge for dashboard display
```

```
    merged = None
```

```
    for name, d in available.items():
```

```
        d_temp = d.copy()
```

```
        if "Student_ID" in d_temp.columns:
```

```
            d_temp["Student_ID"] = d_temp["Student_ID"].astype(str).str.strip()
```

```
            if merged is None:
```

```
                merged = d_temp
```

```
            else:
```

```
                # Merge on Student_ID, avoid duplicate columns
```

```
                new_cols = [c for c in d_temp.columns if c not in merged.columns or c == "Student_ID"]
```

```
                merged = merged.merge(d_temp[new_cols], on="Student_ID", how="outer", suffixes=("", "_dup"))
```

```
    df = merged if merged is not None else pd.DataFrame()
```

```
if df is None or df.empty:
```

```
    st.warning("No data available for dashboard.")
```

```
    st.stop()
```

```
# Calculate CGPA if missing
```

```
if "CGPA" not in df.columns and "Marks" in df.columns:
```

```
    df["CGPA"] = pd.to_numeric(df["Marks"], errors="coerce") / 10
```

```
    df["CGPA"] = df["CGPA"].clip(0, 10).round(2)
```

```
kpis = compute_kpis(df)
```

```
# KPI Cards
```

```
st.markdown("### 📊 Key Performance Indicators")
```

```
col1, col2, col3, col4, col5, col6 = st.columns(6)
```

```

with col1:
    st.markdown(f'<div class="kpi-card"><h3>Total Students</h3><p>{kpis["total_students"]}</p></div>',
                unsafe_allow_html=True)

with col2:
    st.markdown(f'<div class="kpi-card"><h3>Avg Marks</h3><p>{kpis["avg_marks"]}</p></div>',
                unsafe_allow_html=True)

with col3:
    st.markdown(f'<div class="kpi-card"><h3>Avg CGPA</h3><p>{kpis["avg_cgpa"]}</p></div>',
                unsafe_allow_html=True)

with col4:
    st.markdown(f'<div class="kpi-card"><h3>Avg
Attendance</h3><p>{kpis["avg_attendance"]}%</p></div>',
                unsafe_allow_html=True)

with col5:
    st.markdown(f'<div class="kpi-card"><h3>Need
Support</h3><p>{kpis["students_needing_support"]}</p></div>',
                unsafe_allow_html=True)

with col6:
    st.markdown(f'<div class="kpi-card"><h3>Placement
%</h3><p>{kpis["placement_percentage"]}%</p></div>',
                unsafe_allow_html=True)

st.markdown("---")

# Dashboard charts
col_left, col_right = st.columns(2)

with col_left:
    st.markdown("### Department Performance")
    dept_df = department_analysis(df)
    if not dept_df.empty and "Avg_Marks" in dept_df.columns:
        st.bar_chart(dept_df.set_index("Department")["Avg_Marks"])

with col_right:
    st.markdown("### Placement Status")
    if "Placement_Status" in df.columns:
        status_counts = df["Placement_Status"].value_counts()

```

```

        st.bar_chart(status_counts)

st.markdown("---")

col_left2, col_right2 = st.columns(2)

with col_left2:
    st.markdown("### Attendance vs Marks")
    if "Attendance" in df.columns and "Marks" in df.columns:
        plot_df = df[["Attendance", "Marks"]].dropna()
        if not plot_df.empty:
            st.scatter_chart(plot_df, x="Attendance", y="Marks")

with col_right2:
    st.markdown("### Course Performance")
    course_df = course_analysis(df)
    if not course_df.empty and "Avg_Marks" in course_df.columns:
        st.bar_chart(course_df.set_index("Course")["Avg_Marks"])

# Students needing support preview
st.markdown("---")
st.markdown("### 🚨 Students Requiring Academic Support")
support_df = find_students_needing_support(df)
if not support_df.empty:
    st.dataframe(support_df.head(15), use_container_width=True)
    st.caption(f"Showing top 15 of {len(support_df)} students requiring support.")
else:
    st.success("No students currently flagged as requiring academic support.")

#


---


# PAGE: Upload Data
#


---


elif page == "📁 Upload Data":
    st.markdown('<h2 class="section-header">📁 Upload Data</h2>', unsafe_allow_html=True)

```

```
st.markdown("Upload your datasets in **CSV**, **JSON**, or **XML** format.")
```

```
uploaded_files = st.file_uploader(
    "Choose files",
    type=["csv", "json", "xml"],
    accept_multiple_files=True,
    help="Upload one or more data files (admission, registration, attendance, examination, placement)"
)
```

```
if uploaded_files:
```

```
    for uploaded in uploaded_files:
```

```
        with tempfile.NamedTemporaryFile(delete=False, suffix=f"_{uploaded.name}") as tmp:
```

```
            tmp.write(uploaded.read())
```

```
            tmp_path = tmp.name
```

```
        try:
```

```
            df, ftype, msg = detect_and_load(tmp_path)
```

```
            st.success(f"**{uploaded.name}** ( {ftype} ): {msg}")
```

```
        if df is not None and not df.empty:
```

```
            info = get_file_info(df)
```

```
            col1, col2, col3 = st.columns(3)
```

```
            col1.metric("Rows", info["rows"])
```

```
            col2.metric("Columns", info["columns"])
```

```
            col3.metric("Memory", info["memory_usage"])
```

```
            st.dataframe(df.head(10), use_container_width=True)
```

```
        # Determine dataset type from filename
```

```
        fname_lower = uploaded.name.lower()
```

```
        if "admission" in fname_lower:
```

```
            key = "admission"
```

```
        elif "registration" in fname_lower:
```

```
            key = "registration"
```

```
        elif "attendance" in fname_lower:
```

```
            key = "attendance"
```

```

elif "examination" in fname_lower:
    key = "examination"
elif "placement" in fname_lower:
    key = "placement"
else:
    key = uploaded.name.rsplit(".", 1)[0]

st.session_state.datasets[key] = df
st.info(f"Dataset stored as **{key}**.")
except Exception as e:
    st.error(f"Error loading {uploaded.name}: {str(e)}")
finally:
    os.unlink(tmp_path)

# Show currently loaded datasets
st.markdown("---")
st.markdown("### Currently Loaded Datasets")
if st.session_state.datasets:
    for name, df in st.session_state.datasets.items():
        if df is not None and not df.empty:
            st.markdown(f"**{name}**: {len(df)} rows × {len(df.columns)} columns — Columns: {'',
'join(df.columns.tolist())}")
        else:
            st.markdown(f"**{name}**: Empty or not loaded")
    else:
        st.info("No datasets loaded. Use the sidebar button to load sample data.")

#


---


# PAGE: Data Examination
#


---



elif page == "🔍 Data Examination":
    st.markdown('<h2 class="section-header">🔍 Data Examination</h2>', unsafe_allow_html=True)

    if not st.session_state.datasets:
        st.info("Load data first from the sidebar or Upload Data page.")

```

```

st.stop()

dataset_name = st.selectbox("Select Dataset", list(st.session_state.datasets.keys()))
df = st.session_state.datasets.get(dataset_name)

if df is None or df.empty:
    st.warning("Selected dataset is empty.")
    st.stop()

profile = profile_dataset(df, dataset_name)

col1, col2, col3 = st.columns(3)
col1.metric("Total Rows", profile["total_rows"])
col2.metric("Total Columns", profile["total_columns"])
col3.metric("Duplicate Rows", profile["duplicate_rows"])

st.markdown("#### Column Details")
summary_table = create_quality_summary_table(profile)
st.dataframe(summary_table, use_container_width=True)

st.markdown("#### Statistical Summary")
stat_summary = get_statistical_summary(df)
st.dataframe(stat_summary, use_container_width=True)

st.markdown("#### Data Types & Non-Null Info")
basic_info = get_basic_info(df)
st.dataframe(basic_info, use_container_width=True)

st.markdown("#### Data Preview (First 20 Rows)")
st.dataframe(df.head(20), use_container_width=True)

#


---



---


# PAGE: Data Quality
#


---



---



```

```

elif page == "📄 Data Quality":

    st.markdown('<h2 class="section-header"> 📄 Data Quality Report</h2>', unsafe_allow_html=True)

    if not st.session_state.datasets:

        st.info("Load data first from the sidebar.")

        st.stop()

    quality_rows = []

    for name, df in st.session_state.datasets.items():

        if df is None or df.empty:

            quality_rows.append({"Dataset": name, "Rows": 0, "Columns": 0,

                                "Missing Values": "N/A", "Duplicates": 0, "Status": "Empty"})

            continue

        missing_total = int(df.isnull().sum().sum())

        dup_rows = int(df.duplicated().sum())

        dup_ids = int(df["Student_ID"].duplicated().sum()) if "Student_ID" in df.columns else 0

        quality_rows.append({

            "Dataset": name,

            "Rows": len(df),

            "Columns": len(df.columns),

            "Missing Values": missing_total,

            "Duplicate Rows": dup_rows,

            "Duplicate IDs": dup_ids,

            "Status": "Loaded" if len(df) > 0 else "Empty",

        })

    quality_df = pd.DataFrame(quality_rows)

    st.dataframe(quality_df, use_container_width=True)

    # Detailed per-dataset quality

    selected = st.selectbox("Detailed View", list(st.session_state.datasets.keys()))

    df = st.session_state.datasets.get(selected)

    if df is not None and not df.empty:

        st.markdown(f'##### Missing Values in **{selected}**')

```

```

missing_df = pd.DataFrame({
    "Column": df.columns,
    "Missing Count": [int(df[c].isnull().sum()) for c in df.columns],
    "Missing %": [f"{df[c].isnull().mean()*100:.1f}%" for c in df.columns],
})
st.dataframe(missing_df, use_container_width=True)

st.markdown(f'##### Duplicate Analysis in **{selected}**')
st.metric("Exact Duplicate Rows", int(df.duplicated().sum()))
if "Student_ID" in df.columns:
    st.metric("Duplicate Student IDs", int(df["Student_ID"].duplicated().sum()))

#
=====

# PAGE: Data Cleaning
#
=====

elif page == "🔪 Data Cleaning":
    st.markdown('<h2 class="section-header"> 🔪 Data Cleaning</h2>', unsafe_allow_html=True)

    if not st.session_state.datasets:
        st.info("Load data first from the sidebar.")
        st.stop()

    st.markdown("Run the full cleaning pipeline on all datasets: missing value treatment, "
                "duplicate removal, data type correction, and text standardization.")

    if st.button("🔪 Clean All Datasets", type="primary", use_container_width=True):
        cleaned = {}
        treatment_reports = []
        cleaning_logs = []

        for name, df in st.session_state.datasets.items():
            if df is None or df.empty:
                continue

```

```

with st.spinner(f'Cleaning {name}...'):
    cleaned_df, report = full_clean_pipeline(df, name)
    cleaned[name] = cleaned_df

    st.markdown(f'***{name}***: {report['rows_before']} → {report['rows_after']} rows')
    cleaning_logs.append(f'***{name}***: {report['rows_before']} → {report['rows_after']} rows "
        f'(removed {report['rows_before'] - report['rows_after']} rows)')

    if "treatment_report" in report and not report["treatment_report"].empty:
        treatment_reports.append(
            report["treatment_report"].assign(Dataset=name)
        )

    if "duplicate_info" in report:
        di = report["duplicate_info"]
        if di.get("exact_duplicates", 0) > 0:
            st.caption(f' Removed {di['exact_duplicates']} exact duplicate rows')
        if di.get("duplicate_student_ids", 0) > 0:
            st.caption(f' Resolved {di['duplicate_student_ids']} duplicate Student_ID records')

st.session_state.cleaned_datasets = cleaned

st.success("All datasets cleaned successfully!")

# Show treatment summary
st.markdown("### Missing Value Treatment Summary")
if treatment_reports:
    combined_treatment = pd.concat(treatment_reports, ignore_index=True)
    st.session_state.cleaning_treatment = combined_treatment
    st.dataframe(combined_treatment, use_container_width=True)
else:
    st.info("No missing values were found in any dataset.")

# Show cleaned datasets
if st.session_state.cleaned_datasets:
    st.markdown("### Cleaned Datasets")

```

```

sel = st.selectbox("View Cleaned Dataset", list(st.session_state.cleaned_datasets.keys()))
cleaned_df = st.session_state.cleaned_datasets.get(sel)
if cleaned_df is not None and not cleaned_df.empty:
    st.dataframe(cleaned_df.head(20), use_container_width=True)

#


---


# PAGE: Fuzzy Matching
#


---



elif page == "🔗 Fuzzy Matching":
    st.markdown('<h2 class="section-header">🔗 Fuzzy Name Matching</h2>', unsafe_allow_html=True)
    st.markdown("Identify inconsistently entered student names using fuzzy string matching.")

    datasets_to_use = st.session_state.cleaned_datasets if st.session_state.cleaned_datasets else
st.session_state.datasets

    if not datasets_to_use:
        st.info("Load or clean data first.")
        st.stop()

    threshold = st.slider("Similarity Threshold", 60, 95, 85, 5,
        help="Minimum similarity score (0-100) to consider a match")

    selected = st.selectbox("Select Dataset for Fuzzy Matching",
        [k for k, v in datasets_to_use.items()
        if v is not None and not v.empty and "Name" in v.columns])

    if selected:
        df = datasets_to_use[selected]
        names = df["Name"].dropna().unique().tolist()

        with st.spinner("Finding fuzzy matches..."):
            matches = find_fuzzy_matches(names, threshold=threshold)

        if matches:
            fuzzy_df = pd.DataFrame(matches)

```

```

st.session_state.fuzzy_report = fuzzy_df

st.markdown(f"### Found {len(fuzzy_df)} Potential Name Variants")
st.dataframe(fuzzy_df, use_container_width=True)

# Auto-apply standardization
st.markdown("---")

if st.button("✅ Apply Name Standardization (Select Canonical Names)", type="primary"):
    canon_map = create_standardization_map(matches)
    st.markdown(f"**Standardization Map** ({len(canon_map)} variants → canonical names):")
    st.dataframe(pd.DataFrame(list(canon_map.items()), columns=["Variant", "Canonical Name"]),
                  use_container_width=True)

# Apply to all datasets
for name in datasets_to_use:
    d = datasets_to_use[name]
    if d is not None and "Name" in d.columns:
        d["Name"] = d["Name"].map(lambda x: canon_map.get(x, x) if pd.isna(x) else x)

    st.success("Name standardization applied!")
else:
    st.info("No fuzzy matches found above the current threshold.")

#


---


# PAGE: Outlier Analysis
#


---



elif page == "📊 Outlier Analysis":
    st.markdown('<h2 class="section-header">📊 Outlier Analysis</h2>', unsafe_allow_html=True)
    st.markdown("Detect statistical outliers (IQR method) and domain rule violations.")

    datasets_to_use = st.session_state.cleaned_datasets if st.session_state.cleaned_datasets else
st.session_state.datasets

    if not datasets_to_use:
        st.info("Load or clean data first.")

```

```

st.stop()

# Combine all datasets for analysis
all_numeric = []
for name, df in datasets_to_use.items():
    if df is not None and not df.empty:
        for col in ["Marks", "Attendance", "Credits", "CGPA", "Package_LPA"]:
            if col in df.columns:
                numeric_vals = pd.to_numeric(df[col], errors="coerce").dropna()
                if len(numeric_vals) > 0:
                    all_numeric.append({
                        "Dataset": name,
                        "Column": col,
                        "Q1": round(float(numeric_vals.quantile(0.25)), 2),
                        "Q3": round(float(numeric_vals.quantile(0.75)), 2),
                        "IQR": round(float(numeric_vals.quantile(0.75) - numeric_vals.quantile(0.25)), 2),
                        "Lower Bound": round(float(numeric_vals.quantile(0.25) - 1.5 * (numeric_vals.quantile(0.75)
- numeric_vals.quantile(0.25))), 2),
                        "Upper Bound": round(float(numeric_vals.quantile(0.75) + 1.5 * (numeric_vals.quantile(0.75)
- numeric_vals.quantile(0.25))), 2),
                    })

if all_numeric:
    iqr_df = pd.DataFrame(all_numeric)
    st.markdown("### IQR Analysis Summary")
    st.dataframe(iqr_df, use_container_width=True)

# Domain validation
if st.button("🔍 Run Full Outlier Detection", type="primary", use_container_width=True):
    all_reports = []
    all_iqr = {}

    for name, df in datasets_to_use.items():
        if df is not None and not df.empty:
            with st.spinner(f"Detecting outliers in {name}..."):
                report, iqr = detect_and_report_outliers(df, "Student_ID")

```

```

        if not report.empty:
            report["Dataset"] = name
            all_reports.append(report)
            all_iqr[name] = iqr

    if all_reports:
        full_report = pd.concat(all_reports, ignore_index=True)
        st.session_state.outlier_report = full_report
        st.markdown(f"### Outlier Report ({len(full_report)} entries)")
        st.dataframe(full_report, use_container_width=True)

        # Domain rules reference
        st.markdown("### Domain Validation Rules")
        rules_df = pd.DataFrame([
            {"Attribute": k, "Min Valid": v[0], "Max Valid": v[1]}
            for k, v in DOMAIN_RULES.items()
        ])
        st.dataframe(rules_df, use_container_width=True)
    else:
        st.success("No outliers detected.")

#


---


# PAGE: Integrated Dataset
#


---



elif page == "🔗 Integrated Dataset":
    st.markdown('<h2 class="section-header">🔗 Data Integration</h2>', unsafe_allow_html=True)
    st.markdown("Merge all cleaned datasets into one unified student academic dataset using **Student_ID**.")

    datasets_to_use = st.session_state.cleaned_datasets if st.session_state.cleaned_datasets else
    st.session_state.datasets

    if not datasets_to_use:
        st.info("Load or clean data first.")
        st.stop()

```

```

st.markdown("#### Datasets Available for Integration")

for name, df in datasets_to_use.items():
    if df is not None and not df.empty:
        st.markdown(f"- **{name}**: {len(df)} rows × {len(df.columns)} columns")

if st.button("🔗 Integrate All Datasets", type="primary", use_container_width=True):
    with st.spinner("Merging datasets on Student_ID..."):
        integrated, report = integrate_datasets(datasets_to_use)

    if not integrated.empty:
        st.session_state.integrated_df = integrated
        st.success(f"Integration complete!")

        st.markdown(f"- **Rows**: {report.get('total_records_before', 0)} (across all datasets) →  

        **{len(integrated)}** unified records")

        st.markdown(f"- **Columns**: {report.get('total_columns', 0)}")
        st.markdown(f"- **Datasets used**: {' '.join(report.get('datasets_used', []))}")

        # Calculate CGPA if needed
        if "CGPA" not in integrated.columns and "Marks" in integrated.columns:
            integrated["CGPA"] = pd.to_numeric(integrated["Marks"], errors="coerce") / 10
            integrated["CGPA"] = integrated["CGPA"].clip(0, 10).round(2)
            st.session_state.integrated_df = integrated

        st.markdown("### Integrated Dataset Preview")
        st.dataframe(integrated.head(30), use_container_width=True)
    else:
        st.error("Integration failed. Check that datasets contain a Student_ID column.")

elif st.session_state.integrated_df is not None:
    st.markdown("### Current Integrated Dataset")
    df = st.session_state.integrated_df
    st.dataframe(df.head(30), use_container_width=True)

```

#

PAGE: Visualizations

#

```
elif page == "📊 Visualizations":

    st.markdown('<h2 class="section-header">📊 Matplotlib Visualizations</h2>', unsafe_allow_html=True)

    df = st.session_state.integrated_df
    if df is None or df.empty:

        # Fallback to examination data
        if "examination" in st.session_state.datasets and st.session_state.datasets["examination"] is not None:
            df = st.session_state.datasets["examination"]
        else:
            st.info("Please integrate datasets or load data first.")
            st.stop()

    if st.button("📈 Generate All Charts", type="primary", use_container_width=True):
        with st.spinner("Generating visualizations..."):
            charts = generate_all_charts(df)
            st.session_state.charts = charts
            st.success(f"Generated {len(charts)} charts!")

    if st.session_state.charts:
        for name, path in st.session_state.charts.items():
            st.markdown(f"#### {name}")
            st.image(path, use_container_width=True)
            if os.path.exists(path):
                with open(path, "rb") as f:
                    st.download_button(
                        f"📄 Download {name}",
                        f.read(),
                        file_name=f"{name.lower().replace(' ', '_')}.png",
                        mime="image/png",
                    )

    if not st.session_state.charts:
        st.info("Click **Generate All Charts** to create static Matplotlib visualizations.")
```

```
#
```

```
# PAGE: Interactive Analysis
```

```
#
```

```
elif page == "🇮🇹 Interactive Analysis":
```

```
    st.markdown('<h2 class="section-header"> 🇮🇹 Interactive Analysis (Plotly)</h2>', unsafe_allow_html=True)
```

```
    df = st.session_state.integrated_df
```

```
    if df is None or df.empty:
```

```
        st.info("Please integrate datasets first from the **Integrated Dataset** page.")
```

```
        st.stop()
```

```
    # Filters
```

```
    st.markdown("#### 🇮🇹 Filters")
```

```
    col1, col2, col3, col4 = st.columns(4)
```

```
    with col1:
```

```
        departments = ["All"] + sorted([str(d) for d in df["Department"].dropna().unique() if str(d).strip()])
```

```
        dept_filter = st.selectbox("Department", departments)
```

```
    with col2:
```

```
        courses = ["All"] + sorted([str(c) for c in df["Course"].dropna().unique() if str(c).strip()])
```

```
        course_filter = st.selectbox("Course", courses)
```

```
    with col3:
```

```
        if "Semester" in df.columns:
```

```
            sems = ["All"] + sorted([str(int(s)) for s in df["Semester"].dropna().unique()])
```

```
            sem_filter = st.selectbox("Semester", sems)
```

```
        else:
```

```
            sem_filter = "All"
```

```
    with col4:
```

```
        if "Placement_Status" in df.columns:
```

```
            statuses = ["All"] + sorted(df["Placement_Status"].dropna().unique().tolist())
```

```
            status_filter = st.selectbox("Placement Status", statuses)
```

```
        else:
```

```
            status_filter = "All"
```

```

# Apply filters
filtered = df.copy()
if dept_filter != "All":
    filtered = filtered[filtered["Department"] == dept_filter]
if course_filter != "All":
    filtered = filtered[filtered["Course"] == course_filter]
if sem_filter != "All" and "Semester" in filtered.columns:
    filtered = filtered[filtered["Semester"].astype(str) == sem_filter]
if status_filter != "All" and "Placement_Status" in filtered.columns:
    filtered = filtered[filtered["Placement_Status"] == status_filter]

st.markdown(f"***Showing {len(filtered)} records***")

if filtered.empty:
    st.warning("No records match the selected filters.")
    st.stop()

import plotly.express as px
import plotly.graph_objects as go

tab1, tab2, tab3, tab4 = st.tabs(["Marks Distribution", "Department Comparison", "Attendance vs Marks",
"Semester Trend"])

with tab1:
    marks = pd.to_numeric(filtered["Marks"], errors="coerce").dropna()
    if not marks.empty:
        fig = px.histogram(filtered, x="Marks", nbins=25, color="Department",
            title="Marks Distribution (Interactive)",
            hover_data=["Student_ID", "Name", "Department", "CGPA"],
            template="plotly_white")
        fig.update_layout(bargap=0.1)
        st.plotly_chart(fig, use_container_width=True)

with tab2:
    if "Department" in filtered.columns and "Marks" in filtered.columns:
        dept_marks = filtered.groupby("Department")["Marks"].apply(

```

```

        lambda x: pd.to_numeric(x, errors="coerce").mean()
    ).dropna().reset_index()
dept_marks.columns = ["Department", "Avg_Marks"]
if not dept_marks.empty:
    fig = px.bar(dept_marks, x="Department", y="Avg_Marks",
                 color="Department", title="Department-wise Average Marks",
                 template="plotly_white")
    st.plotly_chart(fig, use_container_width=True)

```

with tab3:

```

if "Attendance" in filtered.columns and "Marks" in filtered.columns:
    plot_data = filtered[["Attendance", "Marks", "Student_ID", "Name",
                        "Department", "CGPA"]].dropna()
if not plot_data.empty:
    fig = px.scatter(plot_data, x="Attendance", y="Marks",
                    color="Department", hover_data=["Student_ID", "Name", "CGPA"],
                    title="Attendance vs Marks (Interactive)",
                    template="plotly_white")
    # Add trend line
    fig.add_scatter(x=[plot_data["Attendance"].min(), plot_data["Attendance"].max()],
                   y=[plot_data["Marks"].mean(), plot_data["Marks"].mean()],
                   mode="lines", name="Average Marks",
                   line=dict(color="red", dash="dash"))
    st.plotly_chart(fig, use_container_width=True)

```

with tab4:

```

if "Semester" in filtered.columns and "Marks" in filtered.columns:
    sem_data = filtered.dropna(subset=["Semester"])
    sem_data["Semester"] = pd.to_numeric(sem_data["Semester"], errors="coerce")
    sem_data = sem_data.dropna(subset=["Semester"])

if not sem_data.empty:
    sem_avg = sem_data.groupby("Semester").agg(
        Avg_Marks=("Marks", lambda x: pd.to_numeric(x, errors="coerce").mean()),
        Avg_Attendance=("Attendance", lambda x: pd.to_numeric(x, errors="coerce").mean()),
        Student_Count=("Student_ID", "count"),

```

```

).reset_index()

fig = go.Figure()
fig.add_trace(go.Scatter(x=sem_avg["Semester"], y=sem_avg["Avg_Marks"],
                        mode="lines+markers", name="Avg Marks",
                        line=dict(color="#667eea", width=3)))
if sem_avg["Avg_Attendance"].notna().any():
    fig.add_trace(go.Scatter(x=sem_avg["Semester"], y=sem_avg["Avg_Attendance"],
                            mode="lines+markers", name="Avg Attendance",
                            line=dict(color="#f093fb", width=3, dash="dash")))
fig.update_layout(title="Semester-wise Performance Trend",
                  xaxis_title="Semester", yaxis_title="Value",
                  template="plotly_white")
st.plotly_chart(fig, use_container_width=True)

#


---


# PAGE: Academic Insights
#


---



elif page == "💡 Academic Insights":
    st.markdown('<h2 class="section-header">💡 Academic Insights</h2>', unsafe_allow_html=True)

    df = st.session_state.integrated_df
    if df is None or df.empty:
        st.info("Please integrate datasets first.")
        st.stop()

    # CGPA if missing
    if "CGPA" not in df.columns and "Marks" in df.columns:
        df["CGPA"] = pd.to_numeric(df["Marks"], errors="coerce") / 10
        df["CGPA"] = df["CGPA"].clip(0, 10).round(2)

    # Configurable thresholds
    st.markdown("### ⚙️ Threshold Settings")
    col1, col2, col3 = st.columns(3)

```

```

with col1:
    cgpa_thresh = st.number_input("CGPA Threshold", 0.0, 10.0, 6.0, 0.5)
with col2:
    marks_thresh = st.number_input("Marks Threshold", 0.0, 100.0, 50.0, 5.0)
with col3:
    att_thresh = st.number_input("Attendance Threshold (%)", 0.0, 100.0, 75.0, 5.0)

thresholds = {
    "cgpa_low": cgpa_thresh,
    "marks_low": marks_thresh,
    "attendance_low": att_thresh,
    "low_course_threshold": st.number_input("Low Course Threshold", 0.0, 100.0, 60.0, 5.0),
}

# 1. Students needing support
st.markdown("### 🚨 Students Requiring Academic Support")
support_df = find_students_needing_support(df, thresholds)
if not support_df.empty:
    st.warning(f"{len(support_df)} students require academic support.")
    st.dataframe(support_df, use_container_width=True)
else:
    st.success("No students currently flagged as requiring academic support.")

st.markdown("---")

# 2. Low-performing courses
st.markdown("### 📉 Low-Performing Courses")
low_courses = find_low_performing_courses(df, thresholds["low_course_threshold"])
if not low_courses.empty:
    st.warning(f"{len(low_courses)} courses have average marks below {thresholds['low_course_threshold']}.")
    st.dataframe(low_courses, use_container_width=True)
    st.markdown("**Recommendations:**")
    for _, row in low_courses.iterrows():
        st.markdown(f"- **{row['Course']}** (Avg: {row['Avg_Marks']}): {row['Recommendations']}")
else:

```

```

st.success("No courses are performing below the threshold.")

st.markdown("---")

# 3. Attendance-Performance correlation
st.markdown("### 🇮🇹 Attendance-Performance Analysis")
att_analysis = attendance_performance_analysis(df)
if "error" not in att_analysis:
    col1, col2, col3, col4 = st.columns(4)
    col1.metric("Correlation", f"{att_analysis['correlation']:.4f}")
    col2.metric("High Attendance Avg Marks", f"{att_analysis['high_attendance_avg_marks']}")
    col3.metric("Low Attendance Avg Marks", f"{att_analysis['low_attendance_avg_marks']}")
    col4.metric("Valid Pairs", att_analysis['valid_pairs'])

    st.info(f"**Interpretation**: {att_analysis['interpretation']}")
    st.caption("Note: Correlation does not imply causation. Other factors may influence academic performance.")
else:
    st.warning(att_analysis["error"])

st.markdown("---")

# 4. Department comparison
st.markdown("### 🏢 Department Performance Comparison")
dept_df = department_analysis(df)
if not dept_df.empty:
    st.dataframe(dept_df, use_container_width=True)

st.markdown("---")

# 5. Placement analysis
st.markdown("### 📁 Placement Analysis")
place_info = placement_analysis(df)
if "error" not in place_info:
    col1, col2, col3 = st.columns(3)
    col1.metric("Placement Rate", f"{place_info['placement_percentage']}%")

```

```

col2.metric("Placed", place_info['placed_count'])
col3.metric("Not Placed", place_info['not_placed_count'])

if place_info.get("department_placement") is not None:
    st.markdown("#### Department-wise Placement Rate")
    st.dataframe(place_info["department_placement"], use_container_width=True)

# Performance comparison
st.markdown("#### Placed vs Non-Placed Performance")
comp_data = []
for col in ["Marks", "CGPA", "Attendance"]:
    key_placed = f"avg_{col}_placed"
    key_not = f"avg_{col}_not_placed"
    if key_placed in place_info and key_not in place_info:
        comp_data.append({
            "Metric": f"Avg {col}",
            "Placed": place_info[key_placed],
            "Not Placed": place_info[key_not],
        })
    if comp_data:
        st.dataframe(pd.DataFrame(comp_data), use_container_width=True)

#


---


# PAGE: Download Reports
#


---



elif page == "📄 Download Reports":
    st.markdown('<h2 class="section-header">📄 Download Reports</h2>', unsafe_allow_html=True)

    st.markdown("#### Available Reports for Download")

    col1, col2 = st.columns(2)

    with col1:
        st.markdown("#### 📄 Cleaned Data")

```

```

if st.session_state.cleaned_datasets:

    sel = st.selectbox("Select Cleaned Dataset", list(st.session_state.cleaned_datasets.keys()),
                        key="dl_clean")

    if sel:

        df = st.session_state.cleaned_datasets[sel]

        csv = df.to_csv(index=False).encode("utf-8")

        st.download_button(

            f" 📄 Download Cleaned {sel}.csv",

            csv, f"cleaned_{sel}.csv", "text/csv",

            use_container_width=True

        )

    else:

        st.info("No cleaned datasets available. Run the cleaning pipeline first.")


st.markdown("#### 🔗 Integrated Dataset")

if st.session_state.integrated_df is not None:

    csv = st.session_state.integrated_df.to_csv(index=False).encode("utf-8")

    st.download_button(

        " 📄 Download Integrated Dataset.csv",

        csv, "integrated_dataset.csv", "text/csv",

        use_container_width=True

    )

    else:

        st.info("No integrated dataset available.")


with col2:

    st.markdown("#### 📊 Outlier Report")

    if not st.session_state.outlier_report.empty:

        csv = st.session_state.outlier_report.to_csv(index=False).encode("utf-8")

        st.download_button(

            " 📄 Download Outlier Report.csv",

            csv, "outlier_report.csv", "text/csv",

            use_container_width=True

        )

    else:

```

```

st.info("No outlier report available. Run outlier detection first.")

st.markdown("#### 🔗 Fuzzy Matching Report")
if not st.session_state.fuzzy_report.empty:
    csv = st.session_state.fuzzy_report.to_csv(index=False).encode("utf-8")
    st.download_button(
        "📄 Download Fuzzy Matching Report.csv",
        csv, "fuzzy_matching_report.csv", "text/csv",
        use_container_width=True
    )
else:
    st.info("No fuzzy matching report available.")

st.markdown("---")

# Comprehensive quality report
st.markdown("#### 📄 Comprehensive Data Quality Report")
if st.button("Generate Quality Report", type="primary"):
    report_rows = []
    datasets = st.session_state.cleaned_datasets or st.session_state.datasets
    for name, df in datasets.items():
        if df is not None and not df.empty:
            report_rows.append({
                "Dataset": name,
                "Rows": len(df),
                "Columns": len(df.columns),
                "Missing Values": int(df.isnull().sum().sum()),
                "Duplicate Rows": int(df.duplicated().sum()),
                "Duplicate IDs": int(df["Student_ID"].duplicated().sum()) if "Student_ID" in df.columns else 0,
            })
    if report_rows:
        report_df = pd.DataFrame(report_rows)
        st.dataframe(report_df, use_container_width=True)
        csv = report_df.to_csv(index=False).encode("utf-8")
        st.download_button(

```

```

        "  Download Quality Report.csv",
        csv, "quality_report.csv", "text/csv"
    )

# —— Footer ——
st.sidebar.markdown("---")
st.sidebar.markdown(
    "  *Student Academic Performance*\n"
    "**Data Wrangling & Visualization System**",
)

```

Technologies Used

Python

Pandas → Data manipulation

NumPy → Numerical operations

Matplotlib → Visualization

Regex (re) → Data-format cleaning

RapidFuzz → Fuzzy matching

XML → XML processing

Streamlit → Interactive dashboard

Deliverables

- Integrated and cleaned student dataset from CSV, JSON, and XML sources.
- Data-quality report showing missing values, duplicates, invalid data, and inconsistencies.
- Outlier and fuzzy-matching reports for detecting unusual values and inconsistent student names.
- Matplotlib visualizations for marks, attendance, CGPA, courses, and department performance.
- Interactive academic dashboard with filters and charts.
- Actionable insights identifying students needing support, low-performing courses, attendance-performance relationships, and department trends.
- Downloadable reports and cleaned datasets for academic decision-making.

Conclusion

The **Student Academic Performance Data Wrangling and Visualization System** successfully integrates academic data from CSV, JSON, and XML sources. The system performs data cleaning, standardization, regex validation, fuzzy matching, outlier detection, and data integration. The processed data is then used to generate static and interactive visualizations and derive actionable insights. This system can help universities make better academic decisions through reliable and easily interpretable data.

What I have learnt:

- identifying the data
- removing unwanted data
- cleaning data
- visualizing the data
- creating a website using streamlit package in python
- know how to use the packages Pandas , NumPy , Matplotlib , Regex (re) , Rapid Fuzz
- knowledge about data science domain about queries